

Physics-informed neural networks on six incompressible-flow benchmarks: what the components contribute, where they fail, and what they cost

[Author name(s) and affiliation to be added]

Abstract

Physics-informed neural networks (PINNs) are often presented as mesh-free solvers for fluid flow, but the combination of remedies that makes them train is rarely tested against independent references with its failures reported. We implemented that combination (Fourier features, gated networks with weight factorisation, exact boundary and divergence constraints, causal and gradient-norm loss weighting, adaptive resampling, separable networks and a physics-informed DeepONet) in one JAX code base and ran it on six problems with acceptance criteria fixed in advance. The two-dimensional Taylor–Green vortex was solved to a velocity error of 4.05×10^{-5} and the inverse cylinder-wake problem recovered both equation coefficients to better than 0.1%. On the lid-driven cavity at $\text{Re} = 1000$ the best network is 0.27% from a converged finite-difference solution of the same problem, yet misses a 2% gate against the classical Ghia tables, which an exact solution also misses. Imposing the regularised lid exactly, an impulsive inflow and gradient-norm weighting each produced networks that violate mass or energy conservation, and the cylinder and three-dimensional Taylor–Green gates were not met within the reduced budgets the available GPU time allowed. A physics-informed DeepONet trained over $100 \leq \text{Re} \leq 1000$ predicts the cavity at an unseen $\text{Re} = 400$ with a 9.6% error, passing its 10% gate after three times the planned training. We report all runs (30.5 GPU hours in total), their costs and the reasons for each failure.

1 Introduction

Finite-volume, finite-element and spectral solvers for the incompressible Navier–Stokes equations are fast and well understood for a fixed geometry and fixed parameters. Physics-informed neural networks [1] take another route: a network represents velocity and pressure as smooth functions of space and time, the residuals of the equations are evaluated at scattered points by automatic differentiation, and training drives these residuals and the boundary and initial mismatches towards zero. No mesh is needed, derivatives of the representation are exact, and measurements can enter the same objective, which suits flow reconstruction from sparse data [2] and surrogates queried at arbitrary points.

The optimisation behind this is often badly conditioned.

Networks resolve smooth components first, the loss terms have gradients of very different size, unsteady solutions can be learnt out of temporal order, and convection-dominated problems collapse onto trivial states [3–5]. Many remedies have been proposed [4, 6–18] and collected into practical advice [19], usually each shown on the problem that motivated it.

We test the combination in one code base, built in JAX [20] along the lines of JAX-PI [21] and SPINN [22], on a ladder of problems with gates fixed before training: the two-dimensional Taylor–Green vortex (exact solution), the lid-driven cavity against Ghia et al. [23], the DFG cylinder benchmark of Schäfer and Turek [24], the three-dimensional Taylor–Green vortex at $\text{Re} = 1600$, an inverse problem on the wake data of [1], and a DeepONet over the cavity Reynolds number. Our contributions are: (i) a memory-bounded implementation in which all residuals are built from forward-mode directional derivatives; (ii) an independent finite-difference reference for the regularised-lid cavity, which shows that the 2% Ghia gate cannot be met even by an exact solution, and against which the best network is 0.27% off; (iii) a diagnosis of three ways in which the full recipe produces networks that break conservation: exact lid data incompatible with continuity, an impulsive inflow, and gradient-norm weights of up to 6×10^9 on terms that are already satisfied; (iv) results and costs for every run, including the failed gates and the reduced budgets.

2 Method

Equations. All problems are non-dimensional with $\text{Re} = UL/\nu$. For a network providing velocity $\mathbf{u}$ and pressure p at $z = (t, \mathbf{x})$ the residuals are

$$\mathbf{r}_m = \partial_t \mathbf{u} + \lambda_1 (\mathbf{u} \cdot \nabla) \mathbf{u} + \nabla p - \lambda_2 \nabla^2 \mathbf{u}, \quad r_c = \nabla \cdot \mathbf{u}, \quad (1)$$

with $\lambda_1 = 1$, $\lambda_2 = 1/\text{Re}$ in forward problems and both learnt in the inverse problem. Three output forms are used: velocity–pressure (VP), where r_c is a loss term; a stream function in two dimensions, $u = \partial_y \psi$, $v = -\partial_x \psi$, for which $\nabla \cdot \mathbf{u} \equiv 0$; and a vector potential in three, $\mathbf{u} = \nabla \times \mathbf{A}$, with a weak gauge penalty $10^{-3} \langle (\nabla \cdot \mathbf{A})^2 \rangle$. Every derivative is a forward-mode directional derivative: the Jacobian–vector product of the network with a unit vector e_i gives ∂_i of all outputs, and two nested products give ∂_{ii} . No Jacobian or Hessian tensor is formed (an earlier version that did so needed 63 GB for one stream-function batch),

and the batch is evaluated in rematerialised chunks of 2048 points, so memory grows with the chunk rather than the batch.

Architecture. Periodic coordinates enter as $(\cos \omega x, \sin \omega x)$, which makes the network exactly periodic; other inputs, scaled to $O(1)$, pass through random Fourier features [6] $\gamma(z) = [\cos Bz, \sin Bz]$ with $B_{ij} \sim \mathcal{N}(0, \sigma^2)$ and 128 frequencies ($\sigma = 1$ for the Taylor–Green and cylinder problems, 10 for the cavity, 2 for the wake). The main network is the gated MLP of [3]: two encodings $U = \tanh(W_U \gamma + b_U)$ and $V = \tanh(W_V \gamma + b_V)$ are mixed into every hidden layer, $h_\ell = g_\ell \odot U + (1 - g_\ell) \odot V$ with $g_\ell = \tanh(W_\ell h_{\ell-1} + b_\ell)$, six layers of width 256 unless stated otherwise. Each weight matrix is factorised as $W = V \text{diag}(e^s)$, $s \sim \mathcal{N}(1, 0.1^2)$ [9]. For the three-dimensional problem the network is separable [14], $f_o = \sum_{j=1}^{128} \prod_i f_{o,j}^{(i)}(z_i)$ over the four inputs, with one small network per input; on a tensor grid it is evaluated once per grid line, and a Jacobian–vector product with a vector of ones along one axis gives the exact partial derivative at every grid point. The parametric cavity uses a DeepONet [16, 17], $G(a)(\mathbf{x}) = \sum_{k=1}^{128} b_k(a) \tau_k(\mathbf{x}) + b_0$ with $a = (\log_{10} \text{Re} - 2)/2$, a three-layer branch and a gated six-layer trunk.

Exact constraints and a compatibility condition.

Dirichlet data are built into the ansatz $\hat{\mathbf{u}} = g + \varphi N_\theta$, where g carries the boundary values and φ vanishes on the boundary [12, 13]. For the cavity $\varphi = x(1-x)y(1-y)$ and $g = (u_{\text{lid}}(x)y^8, 0)$ with the regularised lid $u_{\text{lid}} = 1 - \cosh(r(x - \frac{1}{2}))/\cosh(\frac{r}{2})$, $r = 50$; for the channel a bounded product of $\tanh(x/2r)$, $4y(H-y)/H^2$ and $\tanh(d/r^2)$ with d the squared distance to the cylinder rim. The lid profile vanishes at the corners but has slope $\mp r \tanh(r/2) \approx \mp 50$ there. If lid and side walls are both exact, then $\partial_y v = 0$ along the side wall while continuity on the lid demands $\partial_y v = -u'_{\text{lid}}$: no continuously differentiable field satisfies both, and $|\nabla \cdot \hat{\mathbf{u}}| = O(r)$ in corner patches of width $O(1/r)$ whatever the network does. A *soft-lid* variant keeps no-slip and $v = 0$ exact and enforces only the tangential lid velocity through a loss term.

Loss and weighting. The objective is $\sum_i \lambda_i \mathcal{L}_i$ over mean-square residual, boundary, initial-condition, out-flow, data and pressure-anchor terms. Gradient-norm balancing [3, 19] sets $\hat{\lambda}_i = \bar{G}/G_i$ every 1000 steps, with $G_i = \|\nabla_\theta \mathcal{L}_i\|$ and $\bar{G}$ their mean, and smooths $\lambda_i \leftarrow 0.9\lambda_i + 0.1\hat{\lambda}_i$; terms that can vanish exactly (pressure anchor, gauge) keep weight one. Causal weighting [7] sorts the residual points in time, splits them into M chunks with losses $\mathcal{L}_k$ and uses

$$\mathcal{L}_r = \frac{1}{M} \sum_k w_k \mathcal{L}_k, \quad w_k = \exp\left(-\varepsilon \sum_{j < k} \mathcal{L}_j\right), \quad (2)$$

with the weights held constant in the gradient and ε raised through $10^{-2}, \dots, 10^2$ whenever $\min_k w_k > 0.99$. Every

1000 steps residual-based sampling [8] redraws a pool of 65,536 points with probability proportional to $|r|/|\mathbb{E}r| + 1$, keeping 20% uniform.

Training. Adam [25] with a linear warm-up to 10^{-3} and decay by 0.9 every 2000 steps, followed optionally by L-BFGS [26] on one fixed batch. The cavity uses the curriculum $\text{Re} = 100 \rightarrow 400 \rightarrow 1000$ (20k, 40k, 140k steps) and the DeepONet a widening range $\text{Re} \leq 200, 500, 1000$ (40k, 40k, 120k), both carrying the weights over and restarting the optimiser [5]; the DeepONet diverged at a peak rate of 10^{-3} and was trained at 3×10^{-4} with the gradient norm clipped at 1. The cylinder is marched in windows of 0.5 s, each starting from the previous window’s network and final state.

3 Benchmarks, references and protocol

All forward problems are trained on the equations alone; reference data only grade the result. Errors are evaluated off the collocation points, and unsteady pressures are compared after removing the mean in each time slice, since the pressure of an incompressible flow is fixed only up to a function of time.

A: Taylor–Green vortex. The decaying vortex on $[0, 2\pi]^2$ at $\text{Re} = 100$, $t \leq 1$, is an exact solution [27]; the test suite first checks that it gives residuals below 10^{-10} in double precision. Errors use $5 \times 64 \times 64$ space–time points.

B: lid-driven cavity. With the regularised lid of Section 2, the gate compares the centreline profiles $u(0.5, y)$ and $v(x, 0.5)$ at $\text{Re} = 1000$ with the tables of Ghia et al. [23] (relative error at the tabulated stations without the wall points). We transcribed the tables and checked them against the fields distributed with JAX-PI [21], which agree within 5% (2.5% and 1.8% at $\text{Re} = 1000$). Both references use a unit lid, so we also solved the regularised problem with an independent finite-difference code: stream function and vorticity, second-order central differences, Thom’s wall vorticity [28], an exact sine-transform Poisson solve and pseudo-time stepping to $\max |\partial_t \omega| < 10^{-7}$. With a unit lid its primary-vortex minimum of ψ on $128^2, 256^2$ and 512^2 grids ($-0.11547, -0.11807, -0.11872$) extrapolates to -0.11894 , the spectral value of Botella and Peyret [29] (-0.1189366); Ghia et al. report -0.11793 . The 512^2 regularised-lid field is our reference at $\text{Re} = 1000$ (256^2 at 100 and 400).

C: DFG 2D-2 cylinder. Geometry, viscosity and parabolic inflow (mean $\bar{U} = 1$, $\text{Re} = 100$ on the diameter) follow [24]; the outlet carries the do-nothing condition as a loss. Drag and lift come from the surface tractions on 256 points with wall gradients from automatic differentiation;

Table 1: Benchmarks, acceptance gates (fixed before training), budgets (Adam steps + L-BFGS iterations; planned in parentheses) and outcomes. Errors are relative L^2 unless stated.

benchmark	gate	budget used		measured	status
A: Taylor–Green 2D, Re = 100	velocity error $< 10^{-4}$	50k + 1.2k	(50k + 20k)	4.05×10^{-5}	passed
B: cavity, Re = 1000	Ghia u and v errors $< 2\%$	200k + 20k		row C: 0.63% / 2.05% (0.27% from FD field); exact solution: 0.41% / 2.29%	failed
C: DFG 2D-2 cylinder	St within 3% of 0.30, $C_{D,\max}$ within 5% of 3.23	1–2 windows of 4k–12k	(16 × 200k)	no vortex shedding; $C_{D,\max} \leq 1.54$	failed
D: Taylor–Green 3D, Re = 1600	dissipation peak within 5% of 0.01282	80k + 1k	(300k + 20k)	$-dE_k/dt$: 0.0292 at $t = 0$; $2\nu\zeta \leq 4.9 \times 10^{-4}$	failed
inverse wake, Re = 100	recover $\lambda_1 = 1$, $\lambda_2 = 0.01$	25k + 1k	(100k + 20k)	errors 0.004% and 0.065%; hidden pressure 2.44%	passed
PI-DeepONet cavity	error $< 10\%$ at unseen Re = 400	660k	(200k + 20k)	9.65% at Re = 400; 2.1% at 100, 30.0% at 1000	passed

St from the dominant frequency of $C_L(t)$. The 1996 report was not retrievable, so the reference intervals and the drag and lift series come from the FEATFLOW pages [30]; that series gives $St = 0.3002$ and $C_{D,\max} = 3.2201$ (the often quoted $St \approx 0.165$ holds for an unconfined cylinder).

D: Taylor–Green vortex at Re = 1600. The triply periodic vortex [31] becomes turbulent near $t = 8$ –9. We compute E_k and enstrophy ζ on 32^3 grids, the dissipation as $-dE_k/dt$ and as $2\nu\zeta$, and the energy spectrum on 64^3 . The workshop descriptions [32] provide no numerical curves, so we digitised the reference curve of Fig. 4(a) in [33] from a 600 dpi rendering by following the line pixel by pixel: peak 0.01282 at $t = 9.00$; its value at $t = 0$, 4.4×10^{-4} , compares with the exact $3/(4\text{Re}) = 4.69 \times 10^{-4}$.

Inverse problem and parametric model. From the wake data of [1] (5000 points $\times$ 200 snapshots behind an unconfined cylinder) 5000 random velocity samples enter the loss with (1) and unknown $\lambda_{1,2}$, both started at zero; the pressure is never shown and is graded on every tenth snapshot. The DeepONet samples $\text{Re} \in [100, 1000]$ log-uniformly but never inside $[300, 530]$, so $\text{Re} = 400$ is unseen; it is graded against the finite-difference field.

Ablation and budgets. Configuration rows add one component at a time: A plain MLP; B Fourier features; C gated network with weight factorisation; D exact boundary data and divergence; E causal weights; F gradient-norm weights; G adaptive resampling; H separable network (three-dimensional problem only). For the steady cavity E coincides with D. The planned budgets (Table 1) needed about 150 GPU hours; to finish within the time available most runs were cut, as listed. All runs use one seed and one RTX 5070 Ti (16 GB); until the three-dimensional run the card was shared with another workload, which slowed steps by a factor 1.3–1.5.

Table 2: Ablation. A: Taylor–Green 2D after 3k Adam steps (velocity error, maximum divergence). B: cavity at Re = 1000 after the full 200k-step curriculum, end of Adam: Ghia errors and velocity error against the 512^2 finite-difference (FD) field. D and F are given with the exact lid and with the soft lid; E equals D for the steady cavity.

row	A, 3k steps		B, 200k steps		
	error	max div	Ghia u	Ghia v	vs FD
A	8.7×10^{-3}	1.4×10^{-2}	not run at 200k [†]		
B	2.3×10^{-2}	6.4×10^{-2}	not run at 200k [†]		
C	3.2×10^{-2}	7.7×10^{-2}	0.65%	2.06%	0.25%
D exact lid	2.2×10^{-2}	0	21.3%	20.0%	28.3%
D soft lid			1.04%	2.20%	1.9%
E	1.8×10^{-2}	0	(= D)		
F exact lid	1.7×10^{-2}	0	19.5%	19.8%	25.8%
F soft lid			16.0%	15.6%	21.0%
G	1.2×10^{-2}	0	not run at 200k [†]		
FD, same lid			0.41%	2.29%	–

[†]Only run with 16k curriculum steps, a budget at which every row failed (vs FD: A 91%, B 90%, C 81%, D 114%, F 118%, G 113%).

4 Results

4.1 A: two-dimensional Taylor–Green vortex

The full configuration (row G, stream function, 50k Adam steps) reaches a velocity error of 4.05×10^{-5} and a pressure error of 2.59×10^{-4} with zero divergence, passing the gate by a factor of 2.5 (with one gauge constant for the whole space–time domain the same model would show 3.3×10^{-2}). In the short ablation (Table 2) the rows end between 0.9% and 3.2%; at 3k steps the plain network (A) is the most accurate, among the others the error falls steadily from C to G, and the stream-function rows (D–G) are divergence free by construction while the VP rows keep $|\nabla \cdot u|$ of 10^{-2} – 10^{-1} .

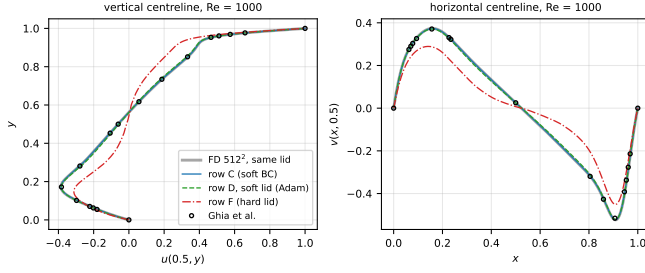


Figure 1: Cavity at $Re = 1000$. Left: centreline velocities of three configurations against the FD solution of the same regularised-lid problem (grey) and Ghia et al. (circles). Right: $|\nabla \cdot u|$ with the exact lid (row F) and with soft boundary data (row C); the exact lid leaves $|\nabla \cdot u| \sim 10$ at the top corners and an unconverged interior.

4.2 B: lid-driven cavity

No configuration passes the 2% gate on v , and the finite-difference reference shows that none can. As the grid is refined the unit-lid solution converges to the spectral primary vortex [29] and moves away from Ghia’s v table (1.7% at 128^2 , 2.1% at 256^2 , 2.8% at 512^2); the converged regularised-lid solution is 2.29% from it, because Ghia’s v values next to the right wall are about 0.01 smaller in magnitude than the converged fields. Row C (soft boundary data, fixed weights) is 0.25% from that field at the end of Adam and 0.27% after L-BFGS, below the 0.56% by which our 256^2 and 512^2 solutions differ (Fig. 1).

The ablation attributes the failures. With the exact lid, rows D and F end 26–28% from the reference whether the weights are fixed or adapted: continuity does not settle below $\sim 10^{-2}$, whereas row C reaches 2×10^{-8} , and the divergence map shows the corner floor predicted in Section 2. The soft lid brings row D to 1.9%. With gradient-norm weighting the soft lid fails again (row F, 21%): the lid term has small gradients, its weight rises to about 465 and makes the lid effectively exact, while the continuity weight falls to 0.2. L-BFGS on one fixed batch of 8192 points is also unsafe here: on the soft-lid row D it lowered that loss from 2.6×10^{-3} to 2.3×10^{-6} and tripled the field error (1.9% to 6.7%).

4.3 C: flow past a cylinder

Four attempts with reduced budgets failed for reasons that have little to do with budget. With the VP network and the impulsive start of the benchmark, drag stayed between 0.3 and 0.5, a tenth of the developed value, and the flow rate through the channel fell to 34% of the inflow at $x = 0.6$ and 2% at the outlet after 0.5 s: the inflow advanced as a slow front. A fluid at rest given the full inflow must accelerate everywhere at once, which a representation smooth in time cannot do, so it trades continuity for smoothness. Ramping the inflow as $\sin^2(\pi t/2)$ over the first second (the periodic state is unaffected) cut the continuity loss 20–80 fold, but still only 12% of the inflow crossed $x = 0.6$: with continuity as one loss term among eight, a small divergence everywhere adds up to a large

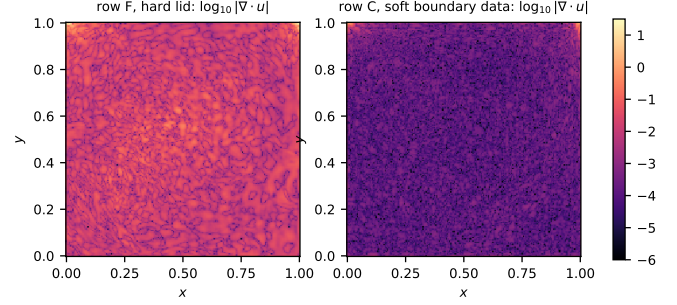


Figure 2: Benchmark D: kinetic energy, the two dissipation estimates of the network against the digitised reference [33], and the spectrum at $t = 9$.

loss of mass along a channel 22 diameters long. A stream-function network with exact divergence and soft wall data kept 75–77% of the flow rate and reached $C_{D,\max} = 1.54$ in two windows (1 s, still inside the ramp), against 3.22 for the developed flow; its remaining loss is leakage through the soft wall conditions. No run reached the shedding regime, so St is undefined and the gate failed.

4.4 D: Taylor–Green vortex at $Re = 1600$

The separable network fits the initial state ($E_k(0) = 0.1246$, exact $1/8$) and then decays smoothly (Fig. 2). $-dE_k/dt$ is largest at $t = 0$ (0.0292) and $2\nu\zeta$ never exceeds 4.9×10^{-4} , errors of 127% and 96% against the reference peak. The two estimates should agree and differ by a factor of about 60: the energy leaves through the momentum residual (mean square $2.5\text{--}7 \times 10^{-4}$), not through viscosity. The vertical velocity stays near zero, so there is no vortex stretching, and the spectrum at $t = 9$ falls as $k^{-3.5}$ rather than $k^{-5/3}$. Gradient-norm weighting gave the initial-condition term for w , whose target is identically zero, a weight of 6×10^9 . The 64^4 grid needed one 11 GB buffer and did not fit, so this run used 32^4 points and a quarter of the planned steps.

4.5 Inverse problem and parametric model

From 5000 velocity samples the wake network recovered $\lambda_1 = 0.99997$ and $\lambda_2 = 0.010006$ (errors 0.004% and 0.065%) and the pressure, never observed, to 2.44% (velocity 0.54% and 2.69%) on 20 snapshots, with a quarter

Table 3: Cost: training wall-clock, Adam step time, peak memory of the training process and inference throughput (batches of 65,536 points). *GPU shared with another workload. ‡0.025 s in the last leg, with a four times larger batch.

run	wall	step [s]	memory	points/s
A, row G	2 h	0.139	–	3.9×10^6
B, row C	1.6 h	0.025	1.6 GB	1.2×10^7
B, row D soft lid*	3.3 h	0.035	1.6 GB	8.9×10^6
C, stream function	1.0 h	0.359	1.9 GB	3.9×10^6
D, row H, grid queries	1.2 h	0.037	3.1 GB	2.0×10^8
inverse	0.5 h	0.055	1.6 GB	4.2×10^6
PI-DeepONet	3.1 h	0.013‡	0.34 GB	1.1×10^7
FD 256 ² / 512 ² , CPU 10 / 57 min				

of the planned budget. The DeepONet diverged twice, both times when the warm-up reached a peak rate of 10^{-3} (once even within the curriculum’s first stage). At 3×10^{-4} with clipping it trained stably: 14.0% from the FD field at the unseen $Re = 400$ after the 200k-step curriculum, then 11.1%, 10.27% and a plateau near 10% in continuations whose learning rate had decayed. Doubling the Reynolds samples and points per step (8 and 8192) at 10^{-5} brought the error to 9.55–9.74% for the last 50k steps. The final model is 9.65% from the FD field at $Re = 400$ (2.1% at 100, 30.0% at 1000) and passes the gate after 660k steps, three times the planned budget; as for the single- Re cavity, the high end of the range is the weak part. A new Re then costs one forward pass.

4.6 Cost

Table 3 lists the cost of the main runs. A trained network answers 3.9×10^6 to 1.2×10^7 point queries per second, and the separable network 2×10^8 on a grid. Training is expensive: the best cavity network took 1.6 GPU-hours, while our deliberately simple finite-difference code reached a comparable accuracy on the CPU in ten minutes. All runs of this study together, including the failed and interrupted ones, used 30.5 GPU hours.

5 Discussion and conclusion

Where the networks did well. With smooth and compatible data the method is accurate: 4×10^{-5} on the two-dimensional vortex and 0.27% of a converged field on the $Re = 1000$ cavity. The inverse problem, in which sparse velocities and unknown coefficients have to be reconciled with the equations, is where the approach is clearly useful: two coefficients to better than 0.1% and a pressure field that was never measured to 2.4%, at a cost of half a GPU-hour. The trained networks are continuous, differentiable functions that can be queried at millions of points per second and sampled at any resolution; the drag integrals, divergence maps and the vortex-tube animation of this study use exactly that.

Where a classical solver wins. For a single forward solve the comparison is not close: our simple finite-difference code produced the cavity reference on a CPU in ten minutes to an hour, while the best network needed 1.6 GPU-hours and other configurations were 20–28% wrong. We did not run a production solver, so no measured comparison with one is claimed.

What the failures have in common. The largest errors of the study share one pattern: the network found a smooth function with small residuals everywhere that nevertheless violates a conservation law. Exact lid data incompatible with continuity left a divergence floor at the cavity corners; an impulsive inflow and, even after a ramp, a long channel let a velocity–pressure network lose most of the mass; and in three dimensions the energy left through the momentum residual instead of through viscosity. Pointwise residual losses do not control these integral quantities, and conservation checks (flow rate through cross-sections, the two dissipation estimates) diagnosed the failures faster than the residuals did. Exact divergence through a stream function restored most of the mass balance. Gradient-norm weighting amplified the problem in every case: it gives each loss term the same share of the update, so terms that are satisfied or easy received weights of 465 (cavity lid), 2.4×10^5 (outflow) and 6×10^9 (initial w), while the hard terms were de-weighted. Terms with an identically zero target should at least be excluded from it.

References and limitations. The Ghia v table is about 2% from grid-converged solutions, so a 2% gate on it cannot be met even by an exact solution: gates should be stated against references that solve the same problem well beyond the required accuracy. Limitations: one seed per configuration; reduced budgets for the cylinder, three-dimensional and inverse runs, and no cylinder ablation; timings 1.3–1.5 times pessimistic for runs on a shared GPU; no 64⁴ separable grid (16 GB); a digitised three-dimensional reference; no OpenFOAM or FEniCSx ground truth.

Conclusion. The representation-side components (Fourier features, gated networks, exact periodicity, stream functions) worked as intended where the data were compatible. The constraint- and weighting-side components need care: exact boundary data fail when the data are incompatible with the equations, and gradient-norm balancing can reinstate that failure through the weights. Three of six gates were passed (A, the inverse problem and the parametric model, the last with three times its planned budget); the cavity gate cannot be met by any exact solution, and the cylinder and three-dimensional gates are out of reach of these budgets. Code, reference solver, logs and evaluation files for every number in this paper are in the accompanying repository.

Acknowledgements. The code and the drafting of this manuscript were assisted by an AI tool (a large language model). All results come from the accompanying repository; responsibility lies with the author(s).

References

- [1] Maziar Raissi, Paris Perdikaris, and George Em Karniadakis. Physics-informed neural networks: A deep learning framework for solving forward and inverse problems involving nonlinear partial differential equations. *Journal of Computational Physics*, 378:686–707, 2019.
- [2] Maziar Raissi, Alireza Yazdani, and George Em Karniadakis. Hidden fluid mechanics: Learning velocity and pressure fields from flow visualizations. *Science*, 367(6481):1026–1030, 2020.
- [3] Sifan Wang, Yujun Teng, and Paris Perdikaris. Understanding and mitigating gradient flow pathologies in physics-informed neural networks. *SIAM Journal on Scientific Computing*, 43(5):A3055–A3081, 2021.
- [4] Sifan Wang, Xinling Yu, and Paris Perdikaris. When and why PINNs fail to train: A neural tangent kernel perspective. *Journal of Computational Physics*, 449:110768, 2022.
- [5] Aditi S. Krishnapriyan, Amir Gholami, Shandian Zhe, Robert M. Kirby, and Michael W. Mahoney. Characterizing possible failure modes in physics-informed neural networks. In *Advances in Neural Information Processing Systems*, volume 34, 2021.
- [6] Matthew Tancik, Pratul P. Srinivasan, Ben Mildenhall, et al. Fourier features let networks learn high frequency functions in low dimensional domains. In *Advances in Neural Information Processing Systems*, volume 33, 2020.
- [7] Sifan Wang, Shyam Sankaran, and Paris Perdikaris. Respecting causality for training physics-informed neural networks. *Computer Methods in Applied Mechanics and Engineering*, 421:116813, 2024. Preprint arXiv:2203.07404 (2022).
- [8] Chenxi Wu, Min Zhu, Qinyang Tan, Yadhu Kartha, and Lu Lu. A comprehensive study of non-adaptive and residual-based adaptive sampling for physics-informed neural networks. *Computer Methods in Applied Mechanics and Engineering*, 403:115671, 2023.
- [9] Sifan Wang, Hanwen Wang, Jacob H. Seidman, and Paris Perdikaris. Random weight factorization improves the training of continuous neural representations, 2022.
- [10] Sifan Wang, Bowen Li, Yuhang Chen, and Paris Perdikaris. PirateNets: Physics-informed deep learning with residual adaptive networks, 2024.
- [11] Xiaowei Jin, Shengze Cai, Hui Li, and George Em Karniadakis. NSFnets (Navier-Stokes flow nets): Physics-informed neural networks for the incompressible Navier-Stokes equations. *Journal of Computational Physics*, 426:109951, 2021.
- [12] I. E. Lagaris, A. Likas, and D. I. Fotiadis. Artificial neural networks for solving ordinary and partial differential equations. *IEEE Transactions on Neural Networks*, 9(5):987–1000, 1998.
- [13] N. Sukumar and Ankit Srivastava. Exact imposition of boundary conditions with distance functions in physics-informed deep neural networks. *Computer Methods in Applied Mechanics and Engineering*, 389:114333, 2022.
- [14] Junwoo Cho, Seungtae Nam, Hyunmo Yang, Seok-Bae Yun, Youngjoon Hong, and Eunbyung Park. Separable physics-informed neural networks. In *Advances in Neural Information Processing Systems*, volume 36, 2023.
- [15] Ameya D. Jagtap and George Em Karniadakis. Extended physics-informed neural networks (XPINNs): A generalized space-time domain decomposition based deep learning framework for nonlinear partial differential equations. *Communications in Computational Physics*, 28(5):2002–2041, 2020.
- [16] Lu Lu, Pengzhan Jin, Guofei Pang, Zhongqiang Zhang, and George Em Karniadakis. Learning nonlinear operators via DeepONet based on the universal approximation theorem of operators. *Nature Machine Intelligence*, 3:218–229, 2021.
- [17] Sifan Wang, Hanwen Wang, and Paris Perdikaris. Learning the solution operator of parametric partial differential equations with physics-informed DeepONets. *Science Advances*, 7(40):eabi8605, 2021.
- [18] Zongyi Li, Nikola Kovachki, Kamyar Azizzadenesheli, Burigede Liu, Kaushik Bhattacharya, Andrew Stuart, and Anima Anandkumar. Fourier neural operator for parametric partial differential equations. In *International Conference on Learning Representations*, 2021.
- [19] Sifan Wang, Shyam Sankaran, Hanwen Wang, and Paris Perdikaris. An expert’s guide to training physics-informed neural networks, 2023.
- [20] James Bradbury, Roy Frostig, Peter Hawkins, et al. JAX: composable transformations of Python+NumPy programs. <https://github.com/google/jax>, 2018.
- [21] Predictive Intelligence Lab. JAX-PI: physics-informed neural networks in JAX. <https://github.com/PredictiveIntelligenceLab/jaxpi>. Accessed September 2026.
- [22] Junwoo Cho and Seungtae Nam. SPINN: separable physics-informed neural networks, reference implementation. <https://github.com/stnamjef/SPINN>. Accessed September 2026.
- [23] U. Ghia, K. N. Ghia, and C. T. Shin. High-Re solutions for incompressible flow using the Navier-Stokes equations and a multigrid method. *Journal of Computational Physics*, 48(3):387–411, 1982.
- [24] M. Schäfer and S. Turek. Benchmark computations of laminar flow around a cylinder. In E. H. Hirschel, editor, *Flow Simulation with High-Performance Computers II*, volume 48 of *Notes on Numerical Fluid Mechanics*, pages 547–566. Vieweg, 1996.
- [25] Diederik P. Kingma and Jimmy Ba. Adam: A method for stochastic optimization. In *International Conference on Learning Representations*, 2015.
- [26] Dong C. Liu and Jorge Nocedal. On the limited memory BFGS method for large scale optimization. *Mathematical Programming*, 45:503–528, 1989.
- [27] G. I. Taylor and A. E. Green. Mechanism of the production of small eddies from large ones. *Proceedings of the Royal Society of London. Series A*, 158(895):499–521, 1937.
- [28] A. Thom. The flow past circular cylinders at low speeds. *Proceedings of the Royal Society of London. Series A*, 141(845):651–669, 1933.
- [29] O. Botella and R. Peyret. Benchmark spectral results on the lid-driven cavity flow. *Computers & Fluids*, 27(4):421–433, 1998.
- [30] FEATFLOW group, TU Dortmund. DFG benchmark 2D-2 (Re=100, periodic): reference drag and lift series. https://wwwold.mathematik.tu-dortmund.de/~featflow/en/benchmarks/cfdbenchmarking/flow/dfg_benchmark2_re100.html. Accessed September 2026.
- [31] Marc E. Brachet, Daniel I. Meiron, Steven A. Orszag, et al. Small-scale structure of the Taylor-Green vortex. *Journal of Fluid Mechanics*, 130:411–452, 1983.
- [32] Z. J. Wang, Krzysztof Fidkowski, Rémi Abgrall, et al. High-order CFD methods: current status and perspective. *International Journal for Numerical Methods in Fluids*, 72(8):811–845, 2013.
- [33] James R. DeBonis. Solutions of the Taylor-Green vortex problem using high-resolution explicit finite difference methods. NASA Technical Memorandum NASA/TM-2013-217850, NASA Glenn Research Center, 2013.